2012 IEEE International Conference on Multimedia and Expo Workshops

Photo Layout with a Fast Evaluation Method and Genetic Algorithm

Jian Fan
Hewlett-Packard Laboratories
Palo Alto, California, USA
jian.fan@hp.com

***Abstract*— Photo collages are an effective form of visual communication and sharing. This paper is devoted to a particular style of photo layout, in which photos are tightly packed into a rectangular canvas while the aspect ratio and orientation of photos are preserved. In addition, users may specify a desired size for each photo within the canvas. BRIC (block recursive image composition) is a method that has been proposed for this problem [1]. In this paper, we present a better method based on the framework of genetic algorithm. Our method builds on three key components. First, we developed a fast method for computing the photo dimensions for a given layout tree. This makes it feasible to evaluate a large number of layout trees within a time limit that is acceptable for interactive multimedia applications. Second, we adapted a fitness measure that takes into account both the photo coverage of the canvas area and the size distribution of photos. Finally, we constructed appropriate genetic operators and incorporated the aforementioned evaluation method and fitness measure. Our experimental results showed that the proposed method is consistently better than BRIC.**

Keywords- Photo layout, genetic algorithm

I. INTRODUCTION

Photo collages are an effective form of visual communication and sharing. For example, a large size photo poster is often used to share travel experiences or other significant events. This form of representation is advantageous in accommodating photos of various sizes and aspect ratios that may not fit well to standard formats such as 4"x6" and 5"x7". The flexibility gives users extra freedom in photo editing that may improve the overall visual appeal. Photo collages come in several different forms and often require manual work. Their automatic creation has attracted the attention of researchers [1][2][3].

For automatic layout of scrapbook-type photo album pages, Geigel and Loui described a system that utilizes a genetic algorithm [2]. Their system allows rotation, scaling and overlap of photos. For the style of the album, they only considered a small number of photos and thus the computational complexity was not a concern. The major technical challenge for automatically creating photo pages of scrapbook style is the lack of a well-established aesthetic measure that is comparable to the eye of human designers.

Another form of photo layout, which is the focus of this paper, is to pack photos tightly into a rectangular canvas with no overlap, possibly with fixed border space between the photos. Photos can be resized but their aspect ratio must be preserved. In addition, a user may specify a desired size distribution for the photos. An example of photos laid out in this format is shown in Figure 1. In this case, one may want the photo of the Golden Gate Bridge to be larger than the others. We believe that automatic creation of this layout type is especially attractive for two reasons. First, due to the inter-photo size constraint of tight packing, it is not straightforward for users to manually create such a layout. Second, the limited design freedom makes the problem more tractable for computer programming.

Fig.1. Ten images of San Francisco in a tightly packed layout for the 13"x19" canvas.

A similar rectangular object packing problem in VLSI circuit layout is called floorplan design [4]. In a widely cited paper [4], Wong and Liu proposed a slicing structure and a simulated annealing method for searching in the solution space. Cohoon et al. proposed a distributed genetic algorithm for the floorplan design problem and compared it against the simulated annealing [5]. Their results showed consistently better results with the genetic algorithm than with simulated annealing.

For the photo layout problem, Atkins proposed the BRIC (block recursive image composition) algorithm [1]. The BRIC algorithm adopts the slicing structure used in the floorplan design. Starting from a single photo, it adds one new photo at a time, such that the new score is highest among all possible photos and layouts. In scoring a particular slicing structure tree, BRIC must solve up to two linear equations of N (the number of photos) variables, which has the complexity of $O(N^3)$. In essence, BRIC is a multi-branch greedy algorithm. As such it has a major limitation in that it can only explore a small solution space limited by the number of branches and the sequential order in which the photos are added. Atkins' earlier work on photo layout dealt with a scrapbook style that allows irregular spacing between photos [6].

In this paper, we present a photo layout method in the framework of genetic algorithm. This includes three major components: 1) a fast computation of a photo layout from a given slicing tree and the associated cost in a linear time of

978-0-7695-4729-9/12 $26.00 © 2012 IEEE
DOI 10.1109/ICMEW.2012.59

$O(N)$, 2) a cost function, and 3) binary slicing tree construction and genetic operators. Our experimental results show consistently better results than BRIC in a comparable amount of running time.

The rest of the paper is organized as follows. Section II gives the problem description and a brief review of the slicing structure and the tree representation. Section III describes the proposed method in detail. Section IV presents experimental results and contrasts them to BRIC. We conclude the paper in Section V with a brief discussion of future work.

II. PRELIMINARIES

The photo layout problem is illustrated in Fig. 1. Given a rectangular canvas S, a set of images $\{p_i|_{0\le i\le(N-1)}\}$ and their desired relative sizes within the canvas $\{t_i|_{0\le i\le(N-1)}\}$, and a *fixed* inter-image space β (the white space in Fig. 1), we want to find an arrangement of the images and their associated scales such that scaled images can be tightly packed into the canvas without overlapping. We assume any necessary image cropping is done by users and the layout algorithm is not allowed to crop any image. For the example of Fig. 1, the canvas is 13x19 (inches), $\beta = 0.025$ (inches), the desired relative sizes are 5 for the Golden Gate Bridge image and 1 for the rest. More precisely, the problem can be described as the following: given a rectangular canvas S of $(W \times H)$, a fixed inter-image space β, and a set of images $P = \{(a_i, t_i)\}_{0\le i\le(N-1)}\}$, where $a_i = w_i/h_i$ and t_i is the aspect ratio and desired relative size of the image i, find an arrangement of the images and their sizes such that a cost function $C(P,S)$ is minimized.

In this paper we consider only the layouts obtainable by a slicing structure that recursively divides a rectangular area into two by either a horizontal or a vertical cut. A slicing structure can be represented by a *full* binary tree. In a full binary tree, each node is either a leaf or has exactly two child nodes. The basic slicing structures and corresponding tree representations are illustrated in Fig. 2. For our photo layout problem, the number of leaf nodes must be equal to the number of photos N. Moreover, given any one of such trees, there are $N!$ ways to assign images to leaves if the aspect ratios of the images are all distinct. For a tree representation with leaves associated with given images, a set of layout parameters, including position and scale for each image, can be computed by solving up to two linear equations of N variables [1].

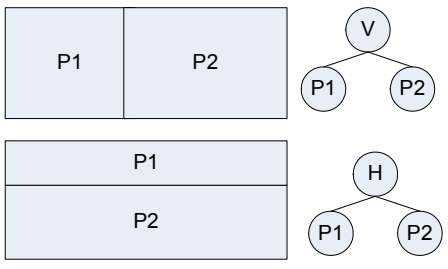

Fig. 2. Slicing structures and corresponding tree representations.

It should be noted that even with the constraint of the slicing structure the problem is still a *NP*-complete combinatorial optimization problem [7]. An exhaustive search is impractical even for a modest number of photos.

III. PROPOSED METHOD

Our method is based on the framework of genetic algorithm. A genetic algorithm mimics a natural evolution process to select and foster the best individuals through generations. Each individual represents a solution to the problem. A genetic algorithm consists of three major components: 1) generation of possible individuals; 2) a metric and a way to measure the goodness of an individual; 3) operators to create new individuals. In order to explore a large solution space, generally a large number of individuals have to be generated and evaluated.

In our case, an individual is a feasible layout represented by a full binary tree with leaf nodes corresponding to given images. The quality of an individual is measured by computing the layout (position and dimension of each image) from a given tree and the cost associated with it. The detail of the proposed method is presented below.

A. Randomized Generation of Layout Trees

Given a set of N images, a feasible tree corresponding to a slicing structure must be a full binary tree with N leaf nodes (denoted L node). According to the property of full binary trees, it must have $(N-1)$ internal nodes (denoted I node). These N leaf nodes are labeled with N labels $\{p_i|_{0\le i\le(N-1)}\}$ and $(N-1)$ internal nodes are labeled with $\{V,H\}$. The I nodes are generated first and the unfilled child nodes are filled with leaves as follows:

1) Create an I node as the root node with a label randomly drawn from the set $\{V,H\}$.
2) Randomly select one I node that has zero or one child from all the I nodes created so far. Create a child I node with a label randomly drawn from $\{V,H\}$, until the total number of I nodes reaches $(N-1)$.
3) For each of the remaining N nodes of the tree, create a L node with a label randomly drawn from the set $\{p_i|_{0\le i\le(N-1)}\}$. Note that the drawn label is not available for the next drawing in order to guarantee the uniqueness of the labels for the L nodes.

B. The Fast Layout Solver

Evaluation of each individual is a key step of the genetic programming. Its computational complexity may determine the running time of an application. For most multimedia applications, a near real-time response is required. Therefore a fast evaluation method is the key to the feasibility of a genetic algorithm approach.

Given a tree representing a feasible layout, corresponding parameters including width and height of every image must be determined and a cost/quality metric can then be computed. Atkins showed that for a given tree the layout of N images can be computed by solving up to two linear

equations of N variables [1]. Solving a linear equation of N variables has a computational complexity of $O(N^3)$. However, we found that the complexity of this approach is too high to be feasible for our genetic algorithm-based method that may involve hundreds of thousands of evaluations.

We observed that the fixed border space β is usually much smaller than photo dimensions. For the purpose of evaluation, a close approximate layout can be obtained with $\beta \equiv 0$. In this case, we found a fast layout solver with a linear complexity $O(N)$. Only for the final solution tree we solve the layout with given β.

Our fast layout solver for the case of $\beta \equiv 0$ is based on the following two observations. Please refer to Fig. 2 for visual illustration.

LEMMA 1. The aspect ratio of a parent rectangle can be computed from the aspect ratio of its two child rectangles with the following formula:

$$\text{"V" node:} \quad a = a_1 + a_2$$
$$\text{"H" node:} \quad 1/a = 1/a_1 + 1/a_2$$

Proof: Assume two child rectangles of aspect ratios a_1 and a_2. For a "V" cut, height and width of the parent rectangle are $h = h_1 = h_2$ and $w = w_1 + w_2$, respectively. This leads to $a = \frac{w_1 + w_2}{h_1} = \frac{(1 + a_2/a_1)w_1}{w_1/a_1} = a_1 + a_2$. The case for the "H" cut can be derived similarly.

This property can be applied recursively to a tree to compute the aspect ratio of the layout. Since the total number of nodes is $(2N-1)$, the complexity of computing the aspect ratio is $O(N)$.

LEMMA 2. Given a canvas of width W and height H and an image of aspect ratio a, the largest image dimensions that can be fit inside the canvas are $w_i = \min(W, aH)$ and $h_i = w_i / a$.

Proof: For the image to fit inside the canvas, its width and height must satisfy the condition $(w_i \le W) \wedge (h_i \le H)$. This leads to $(w_i \le W) \wedge (w_i \le aH)$, or $w_i = \min(W, aH)$.

Again, this property can be applied recursively to a tree in a top-down order and its complexity is also $O(N)$.

Combining the above two properties, the fast layout solver is described in the following *C* pseudo codes on the right side.

Since both *ComputeAR* and *ComputeDimension* have a complexity $O(N)$, the complexity of the fast layout solver is $O(N)$.

```
void SolveLayout(rootNode,width,height) {
    // width and height is the canvas size

    // compute aspect ratio of every I node
    ar_root = ComputeAR( rootNode );

    if( width<ar_root*height )
            w_root = width;
    else
            w_root = ar_root*height;

    h_root = w_root / ar_root;
    // compute width and height of every node
    ComputeDimension(rootNode,w_root,h_root);
}
```

C. The Cost Function

A cost function is a quantitative measure of the quality of an individual. For the photo layout application, we consider two criteria: 1) the coverage of the canvas area by images—a complete coverage is desirable—and 2) the matching of image sizes in the layout to the user's specifications. We want the discrepancies to be as small as possible. Based on the two criteria we design the following cost function:

$$C = \sum_{i=0}^{N-1} k_i (s_i - t_i)^2 + \lambda \left(1 - \sum_{i=0}^{N-1} s_i \right)$$

where $k_i = \begin{cases} 5, & if\ (s_i/t_i) < 0.5 \\ 1, & otherwise \end{cases}$, $s_i = (w_i \cdot h_i)/S$ is the normalized size of the *i-th* image in the layout, S is the size of the canvas, and t_i is the normalized desired size of the *i-th* image specified by the user. For example, if a user specifies desired sizes for 4 images as {1,1,1,5}, the normalized desired sizes will be {0.125,0.125,0.125,0.625}. The first term of the cost function measures the relative error between the actual and desired image sizes. The second term measures the image coverage of the canvas. The trade-off between these two is controlled by the parameter λ.

In contrast, BRIC uses a score function in the form of $R = \sum_{i=0}^{k} \min((s_i/t_i), 1)$ with additional non-linearity branches [1]. It should be noted that this score function does not take canvas coverage into account, and it does not penalize image sizes that are larger than desired. Moreover, there is a fundamental difference between BRIC and our method in terms of the quality evaluation. BRIC is a greedy method. At every step of tree growing, BRIC evaluates not only a new layout but also the choice of an image to add. In our method, the cost function is used to compare two possible solutions with the same complete set of images. Therefore, our cost function may not be applicable to BRIC.

D. The Genetic Operators

Genetic operators are used to generate new individuals from existing ones. Mutation and crossover are two commonly used operators. In our case, the mutation operator swaps the label between a randomly selected node and another randomly selected node of the same type (*I* or *L*) but different label from the same tree. Notice that this operator does not change the structure of a tree.

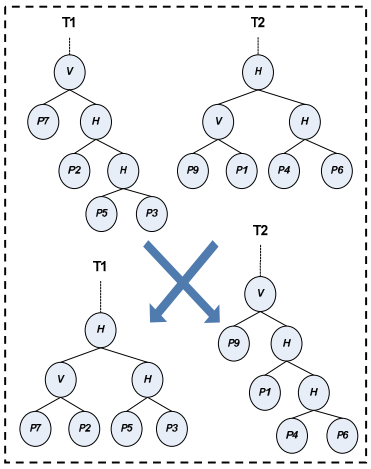

Fig. 3. An example of crossover.

Our crossover operator swaps two subtrees selected from two parent trees. Since the number of images of both trees must be maintained, the two subtrees must have the same number of leaf nodes. Moreover, the labels of the leaf nodes remain in the original tree and are assigned to new *I* nodes. An example of the crossover is illustrated in Fig. 3. Notice that the crossover operation may change the tree structures of both parents. Under this scheme, swapping two subtrees each consisting of one *I* node and two *L* nodes is equivalent to swapping the labels of the two *I* nodes. Therefore, for the crossover, we were only interested in subtrees with at least three *L* nodes. Such a crossover operation may not be carried out for an arbitrary pair of trees since there may not exist two subtrees having the same number of leaf nodes (of at least three). Our crossover operator works as follows:

1) Find all subtrees $\{st_{i,0}, st_{i,1}, st_{i,2}, \ldots\}$ of the tree *i*, and all subtrees $\{st_{j,0}, st_{j,1}, st_{j,2}, \ldots\}$ of the tree *j*.
2) Find all pairs of subtrees $\{st_{i,m}, st_{j,n}\}$ that have the same number of leaf nodes.

If no such pair is found, stop. Otherwise, randomly select a pair of subtrees and perform the crossover as described.

IV. Experimental Results

For genetic programming, we modified the gpc++ version 0.5.2 [8][9], according to Sections III.*A* and III.*D*.

Two sets of 10 and 25 images were used for the experiments. Most of the images were manually cropped to non-standard aspect ratios. The aspect ratios (width/height) for a set of ten images (San Francisco, Fig. 1) are {2.05, 1.53, 1.49, 1.74, 0.54, 1.58, 0.67, 1.20, 2.08, 1.46}. The aspect ratios for a set of 25 images (Hawaii, Fig. 5) are {0.88, 1.33, 1.44, 0.94, 1.84, 1.82, 1.61, 1.35, 1.17, 1.87, 1.65, 1.49, 1.49, 1.73, 1.65, 0.64, 1.91, 0.50, 0.88, 1.74, 1.49, 0.50, 1.70, 1.77, 1.43}.

Furthermore, for the San Francisco image set, the desired sizes are set to 5 for the "Golden Gate Bridge" and 1 for the other 9 images. For the Hawaii image set, the desired sizes are 5 for the "Waipio Valley Lookout" (the largest image in Fig. 5) and 1 for the other 24 images.

For the experimental results presented here, the parameter λ for the cost function *C* was set to 0.15. We record both terms of the cost separately as

$$C_1 = \sum_{i=0}^{N-1} k_i (s_i - t_i)^2 \quad \text{and} \quad C_2 = \left(1 - \sum_{i=0}^{N-1} s_i\right).$$

Our experiments were run on a PC with Intel Core™ 2 X6800 2.93 GHz CPU running Windows® 7.

A. The Effectiveness of the Genetic Algorithm

In this experiment, we used the set of 25 images to assess the layout quality improvement through 300 generations. The population size is 2500. The canvas size is 24x16 (inches^2). The cost terms C_1 and C_2 over the number of generations of genetic evolution recorded in *one run* is plotted in Fig. 4. In this run, the most significant improvements occurred in the first 50 generations and there are several plateaus. The running time is a linear function of the number of generations, as seen in the bottom plot. Due to the randomness involved, the curves of C_1 and C_2 generally vary from run to run. Our experiments suggest that layout quality improvement with C_1 below 0.01 is generally not very significant and this can be used as the stopping criterion.

To visualize the layout improvement, the best layouts of the first and last generation are shown in Fig. 5. It can be seen that sizes of several images in the initial layout are too small and the canvas is left with a significant empty space. In the final layout, the distribution of image sizes is significantly closer to desired and the canvas is almost completely covered.

B. Computational Complexities

The evaluations of layout trees and the overhead of the genetic algorithm are the main contributors to the running time of the algorithm. For this experiment, the algorithm parameters are set as follows. The population is set to 100

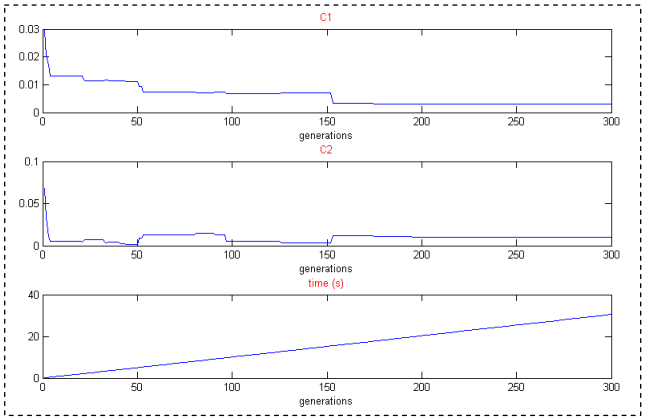

Figure 4. Layout quality improvement through genetic evolutions.

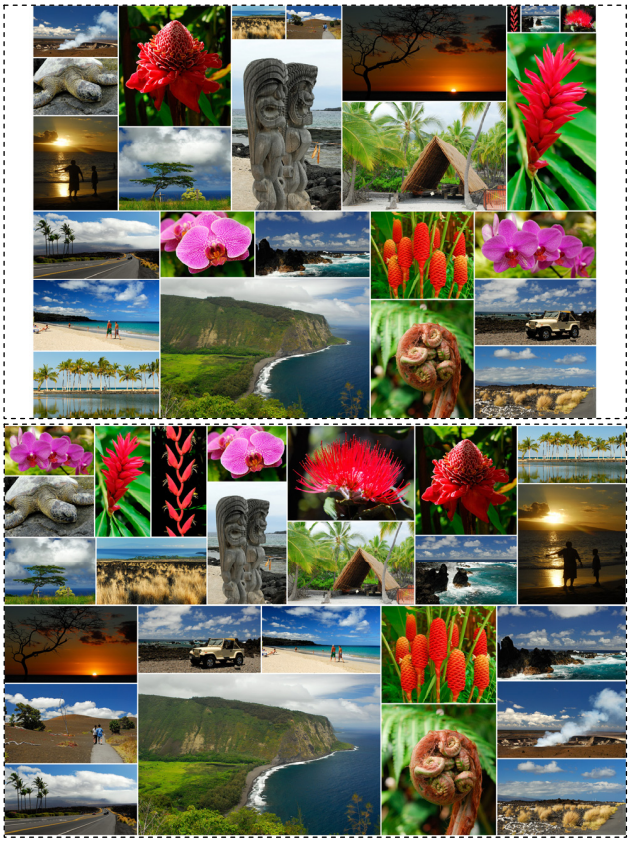

Figure 5. Layout computed after the 1st (top) and 300th (bottom) generations, corresponding to cost terms (C_1,C_2) of (0.0297, 0.0677) and (0.00307, 0.00965), respectively.

times the number of images and the number of generations is set to 40. Five runs on the two sets of images were recorded in terms of total running time T_a and the approximate time spent on evaluation T_e, in Table 1. The timing data suggests that with the fast layout solver, the time spent on evaluation T_e is less than ten percent of the total time. For comparison, we also implemented and tested the exact layout solver using SVD (Singular Value Decomposition) [10]. The timing for five runs with the exact solver is recorded in Table 2. In this case, the total time for 25 images reached nearly one minute, about ninety percent of which was spent on evaluation. Clearly this level of latency is not acceptable for interactive applications.

Table 1. Timing with the fast layout solver

Run	Set of 10 Images		Set of 25 images	
	T_e (s)	T_a (s)	T_e (s)	T_a (s)
1	*0.027*	*0.49*	*0.26*	*4.21*
2	*0.039*	*0.49*	*0.25*	*4.13*
3	*0.036*	*0.48*	*0.28*	*4.10*
4	*0.042*	*0.47*	*0.25*	*4.23*
5	*0.039*	*0.48*	*0.24*	*4.19*
mean	*0.037*	*0.48*	*0.26*	*4.17*

Table 2. Timing with the exact solver

Run	Set of 10 Images		Set of 25 images	
	T_e (s)	T_a (s)	T_e (s)	T_a (s)
1	*2.57*	*2.98*	*53.0*	*56.9*
2	*2.39*	*2.92*	*53.1*	*56.9*
3	*2.66*	*3.01*	*52.9*	*56.8*
4	*2.48*	*2.92*	*52.9*	*57.1*
5	*2.50*	*2.96*	*53.5*	*57.5*
mean	*2.52*	*2.96*	*53.1*	*57.0*

C. *Comparison with BRIC*

We benchmarked our results with BRIC. For the reason explained in Section 3.3, we cannot apply our cost function to BRIC. Therefore, we treated BRIC as a black box and used our cost function to evaluate its final layouts. For experiments, BRIC and the proposed method were both given the same images and the same layout specifications, including canvas dimensions and desired size distribution. For BRIC, the number (L) of trees to carry forward was set to 6. For the proposed method, the population was set to 100 times the number of images and the number of generations was set to 40.

Table 3 presents the results of five consecutive runs using the 10-image set of San Francisco with canvas dimensions of 13x19 (inches^2) and the aforementioned desired size distribution. The cost metrics indicate that the layouts generated by the proposed method are significantly better and more consistent than those generated by BRIC.

Table 4 shows the results of five consecutive runs using the 25-image set of Hawaii with canvas dimensions of 24x16 (inches^2) and the aforementioned size distribution. Again, the proposed method generated significantly better and more consistent layouts than BRIC within a comparable running time.

Table 3. Algorithm comparison with the 10-image set

Run	BRIC			PROPOSED		
	C_1	C_2	*T(s)*	C_1	C_2	*T(s)*
1	0.32	0.23	0.16	0.0065	0.035	0.49
2	0.56	0.30	0.17	0.0058	0.038	0.49
3	0.022	0.19	0.16	0.0098	0.090	0.48
4	0.56	0.30	0.16	0.0200	0.048	0.47
5	0.058	0.03	0.17	0.0054	0.047	0.48
mean	0.30	0.21	0.16	0.0096	0.052	0.48

Table 4. Algorithm comparison with the 25-image set

Run	BRIC			PROPOSED		
	C_1	C_2	*T(s)*	C_1	C_2	*T(s)*
1	0.092	0.068	3.8	0.0092	0.034	4.2
2	0.12	0.032	3.8	0.018	0.039	4.1
3	0.18	0.065	3.8	0.014	0.027	4.1
4	0.13	0.065	3.8	0.012	0.037	4.2
5	0.0071	0.26	3.8	0.011	0.035	4.2
mean	0.092	0.099	3.8	0.013	0.035	4.2

Figure 6. Best layouts generated by BRIC (top) and the proposed method (bottom), corresponding to cost terms (C_1, C_2, C) of (0.0071, 0.2636, 0.073) and (0.0092, 0.034, 0.016), respectively.

It should be noted that the runtime of each method depends not only on the complexity of the layout solver, but also the number of evaluations, the size of the tree for each evaluation, and the overhead in generating the candidate trees. For BRIC, the number of evaluations is determined by the number of trees (L) carried forward while the size of trees varies at different stages. For the proposed method, the number of evaluations depends on the size of the population and the number of generations of evolution; the size of trees for evaluation remains the same and is determined by the number of images. The proposed method is advantageous in controlling the trade-off between runtime and layout quality. While the maximum runtime can be guaranteed by choosing the population size and the maximum number of generations, an additional early stopping criterion based on the cost measure (such as C_1<0.02) can be incorporated. Genetic evolution will stop when either the cost criterion or the specified maximum number of generations is reached.

For visualizing layouts generated using BRIC and the proposed method, Fig. 6 presents the best layouts from the five runs using both methods. In the case of BRIC, although the image size distribution is quite satisfactory, the canvas coverage is poor. The layout generated by the proposed method is able to satisfy both measures and reached a significantly lower overall cost.

V. Conclusion

In this paper, we present a photo layout method in the framework of genetic algorithm and utilizing a slicing structure. We show an approximate layout solver with a linear complexity $O(N)$. We also designed a cost function and two genetic operators. Our experimental results show that the proposed algorithm significantly outperformed BRIC in a comparable amount of running time. Our experimental data also suggest that with the fast evaluation method, the overhead of genetic programming becomes dominant and should be addressed.

In future work, we plan to incorporate other features, such as image positions, into the cost function. We also plan to investigate other types of genetic operators as well as the overhead of the genetic programming framework.

Acknowledgment

The author would like to thank Dr. Brian Atkins and Dr. Hui Chao for BRIC codes.

References

[1] C. Atkins, "Blocked recursive image composition," Proc. ACM MM, 2008, pp. 821-824, doi: 10.1145/1459359.1459496

[2] J. Geigel and A. Loui. "Using genetic algorithms for album page layouts," *IEEE Multimedia*, vol. 10, no. 4, 2003, pp. 16-26, doi: 10.1109/MMUL.2003.1237547

[3] J. Kustanowitz and B. Shneiderman. "Hierarchical layouts for photo libraries," *IEEE Multimedia*, vol. 13, no. 4, 2006, pp. 62-72, doi: 10.1109/MMUL.2006.83

[4] D. F. Wong, and C. L. Liu, "A new algorithm for floorplan design," *Proc. DAC*, 1986, pp.101-107.

[5] James P. Cohoon, et al, "Distributed genetic algorithms for the floorplan design problem," *IEEE Trans. CAD*, vol. 10, no. 4, 1991, pp. 483-492, doi: 10.1109/43.75631

[6] C. Atkins, "Adaptive photo collection page layout," *Proc. ICIP*, 2004, pp. 2897-2900, doi: 10.1109/ICIP.2004.1421718

[7] Bo Yao, et al, "Floorplan representations: Complexity and connections", *ACM Transactions on Design Automation of Electronic Systems*, Vol. 8, No. 1, 2003, pp. 55–80, doi: 10.1145/606603.606607

[8] A. P. Fraser, "Genetic programming in C++", GPC++ version 0.40, 1994

[9] A. P. Fraser and T. Weinbrenner, "The genetic programming kernel," Version 0.5.2, 1997

[10] W. H. Press, *et al*, Numerical recipes in C, 2nd Edition, Cambridge University Press